



(/rol/app/)

Menu

Korean

카탈로그 (/rol/app/catalog) (catalog) > Observing OpenShift Service Mesh

> Chapter 1: Observing OpenShift Service Mesh

Observing OpenShift Service Mesh

Lesson

Lab Environment

Guided Exercise: Collecting Service Metrics

Instrument Node.js and Java applications to expose custom counter and gauge metrics.

Configure ServiceMonitor resources to collect custom application metrics in OpenShift user workload monitoring.

Query and analyze service mesh metrics and custom application metrics by using the OpenShift web console.

Outcomes

- Configure Red Hat OpenShift Service Mesh (OSSM) projects for monitoring.
- Create custom metrics in a Java Quarkus application and a Javascript Node.js application.
- Configure metrics monitoring for your services in the mesh.
- Inspect service metrics by using the OpenShift web console monitoring tools.

As the student user on the workstation machine, use the lab command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
[student@workstation ~]$ lab start meshobservability-metrics
```

Also, run the following command to update your \$PATH variable and make the traffic_gen.py command available immediately. Run it once after creating a new environment.

```
[student@workstation ~]$ source ~/.bashrc
```



In this exercise, you create and visualize custom metrics for the following applications:

product

Is a JavaScript application that uses the Node.js runtime. This service renders product details available for sale in the store. Its main endpoint is at the `/spl50` URL.

order

Is a Java application that uses the Quarkus framework. This service handles customer orders. It has two endpoints: `/orders` and `/rating`.

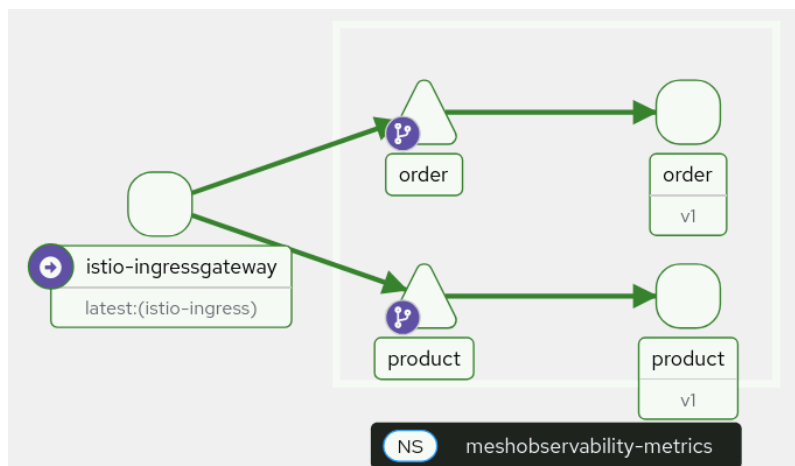


Figure 1.27: Traffic graph for the two applications, after including them in the mesh.

You can find the solution files for this exercise in the `~/course/solutions` directory.

Instructions

1. Add the custom metrics to the product Node.js application, and deploy it to OpenShift.

This microservice exposes an HTTP GET endpoint named `/spl50` that renders a 50% discount offer page to customers. It depends on the `prom-client` library to send metrics to Prometheus.

You create the following metrics:

- `product_svc:spl50_response_time`: Response time to render the response for the `/spl50` endpoint.
- `product_svc:spl50_page_view_count`: Page view count for the `/spl50` endpoint.

- 1.1. In the workstation machine, change to the exercise working directory, and review the source code for the product microservice in the `server.js` file.

```
[student@workstation ~]$ cd ~/course/labs/meshobservability-metrics/product
```

- 1.2. Inspect the `package.json` to know all the Node.js Package Manager (NPM) packages required for running this microservice, and build the dependencies.

```
[student@workstation product]$ cat package.json
{
  "name": "product",
  "version": "1.0.0",
  ...output omitted...
  "dependencies": {
    "express": "^4.16",
    "prom-client": "^13.2.0"
  },
  ...output omitted...
```

The product microservice depends on the prom-client Prometheus library to send metrics.

Build the dependencies:

```
[student@workstation product]$ npm install
added 72 packages, and audited 73 packages in 990ms
...output omitted...
```

- 1.3. Modify the server.js file to import the prom-client package and initialize it.

Add a unique prefix named product_svc_ to identify the standard Node.js runtime metrics for this application in Prometheus.

```
...output omitted...
const express = require('express');
const prometheus = require('prom-client');
const prefix = 'product_svc_';
prometheus.collectDefaultMetrics({ prefix });
```

- 1.4. Add code to create a Prometheus gauge type for tracking response time. Note the use of the service name as a prefix to identify this custom metric in Prometheus.

```
...output omitted...
app.listen(8080, function () {
  console.log('product-svc started on port 8080');
})

const responseTime = new prometheus.Gauge({
  name: 'product_svc:spl50_response_time',
  help: 'Time take in seconds to render the 50% special offer page'
});
...output omitted...
```

- 1.5. Add code to create a Prometheus counter type for tracking the page view count.

```

...output omitted...
app.listen(8080, function () {
    console.log('product-svc started on port 8080');
})

const responseTime = new prometheus.Gauge({
    name: 'product_svc:spl50_response_time',
    help: 'Time take in seconds to render the 50% special offer page'
});

const page_views = new prometheus.Counter({
    name: 'product_svc:spl50_page_view_count',
    help: 'No of page views for the 50% special offer page'
});
...output omitted...

```

- 1.6. Edit the code for the /spl50 route handler to start the timer, which measures the response time as soon as the function begins to execute. Increment the page view counter.

```

...output omitted...
app.get('/spl50', async function (req, res) {
    responseTime.setToCurrentTime();
    const end = responseTime.startTimer();
    page_views.inc();
...output omitted...

```

- 1.7. Before sending the response to the client, end the timer that you started at the beginning of the function, after the sleep() call.

```

...output omitted...
await sleep(Math.floor(Math.random() * 200) + 1);
end();
res.send(view_msg);
...output omitted...

```

- 1.8. Add code to create a /metrics endpoint.

Prometheus uses this endpoint to collect metrics for this microservice.

```

...output omitted...
app.get('/metrics', async function (req, res) {
    res.set('Content-Type', prometheus.register.contentType);
    res.send(await prometheus.register.metrics());
})

```

- 1.9. Test the changes in the code by running the service on the workstation machine.

```

[student@workstation product]$ npm install
...output omitted...
[student@workstation product]$ node server.js
product-svc started on port 8080

```

Open a new terminal and verify that the `/metrics` endpoint shows all the metrics that the `prom-client` library exposes.

```
[student@workstation product]$ curl http://localhost:8080/metrics
# HELP product_svc_process_cpu_user_seconds_total Total user CPU time spent in seconds.
# TYPE product_svc_process_cpu_user_seconds_total counter
product_svc_process_cpu_user_seconds_total 1.825365

# HELP product_svc_process_cpu_system_seconds_total Total system CPU time spent in seconds.
# TYPE product_svc_process_cpu_system_seconds_total counter
product_svc_process_cpu_system_seconds_total 0.269166

# HELP product_svc_process_cpu_seconds_total Total user and system CPU time spent in seconds.
# TYPE product_svc_process_cpu_seconds_total counter
product_svc_process_cpu_seconds_total 2.094531
...output omitted...
```

Press **Ctrl+C** to stop the server.

2. Add the custom metrics to the order Java Quarkus application, and deploy it to OpenShift.

The following are the new metrics:

- Response time to process an order.
- Count of orders placed after viewing the special offer page.
- Satisfaction rating for the order process.

- 2.1. Change to the `~/course/labs/meshobservability-metrics/order` directory, and inspect the `maven pom.xml` file, which declares the dependencies for this microservice.

```
[student@workstation product]$ cd ~/course/labs/meshobservability-metrics/order
```

- 2.2. Edit the `pom.xml` file and include the `quarkus-micrometer` and `quarkus-micrometer-registry-prometheus` dependencies for enabling metrics collection.

Add these dependencies after the `quarkus-smallrye-health` dependency.

```

...output omitted...
<dependency>
  <groupId>io.quarkus</groupId>
  <artifactId>quarkus-micrometer</artifactId>
</dependency>
<dependency>
  <groupId>io.quarkus</groupId>
  <artifactId>quarkus-micrometer-registry-prometheus</artifactId>
</dependency>
...output omitted...

```

- 2.3. Edit the `src/main/java/com/redhat/training/order/OrderService.java` file to modify the annotations for the `processOrder()` method.

Add a counter that tracks the number of orders placed.

```

...output omitted...
@GET
@Path("/order")
@Produces(MediaType.TEXT_PLAIN)
@Counted(value = "orders_placed",
  description = "count of spl50 orders placed")
...output omitted...
public String processOrder() {
...output omitted...

```

- 2.4. Add a timer to track the response time for processing an order.

```

...output omitted...
@Counted(value = "orders_placed", description = "count of spl50 orders
placed")
@Timed(value = "order_process_time",
  description = "A measure of how long it takes to process an or
der")
...output omitted...

```

- 2.5. Add a counter type metric to the `generateRandomRating()` method to capture the rating for the order.

```

...output omitted...
@Counted(value = "order_process_rating",
  description = "Overall customer rating for the order proce
ss")
Integer generateRandomRating() {
...output omitted...

```

- 2.6. Compile the application and test the changes in the code by running the service on the workstation machine.

First, run `mvn clean package` to package the microservice.

NOTE

The first time you compile the application, Maven spends more than 15 minutes downloading dependencies.

```
[student@workstation order]$ mvn clean package
...output omitted...
[INFO] [io.quarkus.deployment.pkg.steps.JarResultBuildStep] Building uber jar: /home/student/course/labs/meshobservability-metrics/order/target/order-1.0.0-runner.jar
...output omitted...
[INFO] [io.quarkus.deployment.QuarkusAugmentor] Quarkus augmentation completed in 4923ms
[INFO] -----
[INFO] BUILD SUCCESS
...output omitted...
```

Then, start the microservice and verify that there are no errors.

```
[student@workstation order]$ java -jar target/order-1.0.0-runner.jar
...output omitted...
2025-10-10 06:56:12,632 INFO [io.quarkus] (main) order 1.0.0 on JVM (powered by Quarkus 3.15.7.redhat-00001) started in 1.027s. Listening on: http://0.0.0.0:8080
...output omitted...
```

Open a new terminal and review the `/q/metrics` endpoint to see all the metrics that Quarkus exposes.

```
[student@workstation ~]$ curl http://localhost:8080/q/metrics
# TYPE jvm_gc_overhead gauge
# HELP jvm_gc_overhead An approximation of the percent of CPU time used by GC activities over the last lookback period or since monitoring began, whichever is shorter, in the range [0..1]
jvm_gc_overhead 2.3282173135511473E-4
# TYPE jvm_buffer_total_capacity_bytes gauge
# HELP jvm_buffer_total_capacity_bytes An estimate of the total capacity of the buffers in this pool
jvm_buffer_total_capacity_bytes{id="mapped - 'non-volatile memory'"} 0.0
jvm_buffer_total_capacity_bytes{id="mapped"} 0.0
jvm_buffer_total_capacity_bytes{id="direct"} 1429557.0
# TYPE process_files_open_files gauge
# HELP process_files_open_files The open file descriptor count
process_files_open_files 16.0
...output omitted...
```

The metrics that you added are still not present because the application has no traffic yet. Press **Ctrl+C** to stop the server.

Deploy the applications to OpenShift, add the project to the mesh, and apply the gateway and virtual service mesh resources to expose the applications. You can deploy the applications as the developer user, but adding them to the mesh requires higher permissions.

3.1. Package and deploy the product service by using the provided script.

From the `~/course/labs/meshobservability-metrics` directory, log in to OpenShift, switch to the `meshobservability-metrics` project, and run the `package_and_deploy_product.sh` script. This script builds the container image, pushes the image to the classroom registry, and deploys the service by using the `product-deploy.yaml` OpenShift resources.

```
[student@workstation order]$ cd ~/course/labs/meshobservability-metrics
[student@workstation meshobservability-metrics]$ oc login -u developer \
-p developer https://api.ocp4.example.com:6443
Login successful.
...output omitted...
[student@workstation meshobservability-metrics]$ oc project meshobservability-metrics
...output omitted...
[student@workstation meshobservability-metrics]$ ./package_and_deploy_product.sh
Login Succeeded!
STEP 1/6: FROM registry.access.redhat.com/ubi8/nodejs-12
...output omitted...
Successfully tagged registry.ocp4.example.com:8443/developer/ossm-metrics-product:1.0
...output omitted...
deployment.apps/product created
service/product created
serviceaccount/product created
```

Finally, verify that the product service is running:

```
[student@workstation meshobservability-metrics]$ oc get pods
NAME                                READY   STATUS    RESTARTS   AGE
product-64c7f7fd68-jh9b5           1/1     Running   0           45s
```

3.2. Package and deploy the order service by using the provided script.

Run the `package_and_deploy_order.sh` script. This script builds the container image, pushes the image to the classroom registry, and deploys the service by using the `order-deploy.yaml` OpenShift resources.


```
[student@workstation meshobservability-metrics]$ ./package_and_deploy_order.sh
Login Succeeded!
STEP 1/6: FROM registry.access.redhat.com/ubi8:8.6
...output omitted...
Successfully tagged registry.ocp4.example.com:8443/developer/ossm-metrics-order:1.0
...output omitted...
deployment.apps/order created
service/order created
serviceaccount/order created
```

Finally, verify that the order service is running.

```
[student@workstation meshobservability-metrics]$ oc get pods
NAME                                READY   STATUS    RESTARTS   AGE
order-674fd6496c-xd2sm             1/1     Running   0           5s
product-64c7f7fd68-jh9b5          1/1     Running   0          45s
```

Note that each pod has only one container.

- 3.3. Add the `istio-injection=enabled` label to the `meshobservability-metrics` namespace.

By default, OpenShift does not permit regular users to add labels to namespaces. Thus, log in as the admin user and the `redhatocp` password:

```
[student@workstation meshobservability-metrics]$ oc login -u admin \
-p redhatocp https://api.ocp4.example.com:6443
Login successful.
...output omitted...
```

Add the label to the namespace:

```
[student@workstation meshobservability-metrics]$ oc label namespace \
meshobservability-metrics istio-injection=enabled
namespace/meshobservability-metrics labeled
```

Log in as the developer user and the developer password, switch to the `meshobservability-metrics` project:

```
[student@workstation meshobservability-metrics]$ oc login -u developer \
-p developer https://api.ocp4.example.com:6443
Login successful.
...output omitted...
[student@workstation meshobservability-metrics]$ oc project meshobservability-metrics
...output omitted...
```

Then, rollout the two deployments objects to make Istio inject the sidecar containers:

```
[student@workstation meshobservability-metrics]$ oc rollout \
restart deployment order
deployment.apps/order restarted
[student@workstation meshobservability-metrics]$ oc rollout \
restart deployment product
deployment.apps/product restarted
[student@workstation ~]$
```

3.4. Apply the ingress Gateway resource.

```
[student@workstation meshobservability-metrics]$ oc apply -f gateway.y
aml

gateway.networking.istio.io/meshobservability-metrics-gateway created
```

This gateway exposes the applications through the common `istio-ingressgateway`. Thus, the common URL of both microservices starts with `http://istio-ingressgateway-istio-ingress.apps.ocp4.example.com/`.

3.5. Apply the VirtualService object to route traffic to the product and order services.

```
[student@workstation meshobservability-metrics]$ oc apply \
-f order-product-vs.yaml
virtualservice.networking.istio.io/order-product-vs created
```

4. Add the monitors to collect all the metrics from both applications, and generate traffic to the endpoints.

Configuring the monitoring is also a privileged operation in an OpenShift cluster. Thus, identify yourself as `admin` and apply the service monitors to enable metrics scraping for the two services.

4.1. Inspect the service monitors files for order, the `order-monitor.yaml` file, and for product, the `product-monitor.yaml` file.

```
[student@workstation meshobservability-metrics]$ cat product-monitor.yaml
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: product-monitor
  namespace: meshobservability-metrics
spec:
  endpoints:
    - interval: 30s
      path: /metrics
      scheme: http
      targetPort: 8080
  selector:
    matchLabels:
      app: product

[student@workstation meshobservability-metrics]$ cat order-monitor.yaml
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: order-monitor
  namespace: meshobservability-metrics
spec:
  endpoints:
    - interval: 30s
      path: /q/metrics
      scheme: http
      targetPort: 8080
  selector:
    matchLabels:
      app: order
```

These service monitors define the URL path in the service to scrape the metrics, the service port, and the label selector for each microservice. Note that the Quarkus application exposes its metrics in `/q/metrics` endpoint, instead of the most common `/metrics`.

- 4.2. Create the service monitors by applying them from the exercise working directory.

First, log in as the admin user.

```
[student@workstation meshobservability-metrics]$ oc login -u admin \
-p redhatocp https://api.ocp4.example.com:6443
Login successful.
...output omitted...
```

Then, apply the service monitors.

```
[student@workstation meshobservability-metrics]$ oc apply \
-f product-monitor.yaml
servicemonitor.monitoring.coreos.com/product-monitor created
[student@workstation meshobservability-metrics]$ oc apply \
-f order-monitor.yaml
servicemonitor.monitoring.coreos.com/order-monitor created
```

4.3. Generate traffic to the endpoints.

Open a different terminal on the workstation machine and run the `traffic_gen.py` utility to generate traffic to each endpoint. This script uses the `traffic_order_and_product.yaml` file as the configuration for generating HTTP traffic.

```
[student@workstation ~]$ cd ~/course/labs/meshobservability-metrics

[student@workstation meshobservability-metrics]$ traffic_gen.py \
traffic_order_and_product.yaml
🔧 Mix mode: pattern=round-robin, items=3
🔗 Create orders: curl -s http://istio-ingressgateway-istio-ingress.ap
pps.ocp4.example.com/order
🔗 Product offers: curl -s http://istio-ingressgateway-istio-ingress.a
pps.ocp4.example.com/spl50
🔗 Get ratings: curl -s http://istio-ingressgateway-istio-ingress.app
s.ocp4.example.com/rating
[1] Create orders ✅ HTTP 200 -- Thank you for your order! Your order
id is 9375 (0.2s)
[2] Product offers ✅ HTTP 200 -- 50% off on purchase of 100 or more i
tems!
Hurry! Limited stocks... (0.9s)
[3] Get ratings ✅ HTTP 200 -- You rated the order process 5 stars. Th
ank you for your feedback! (1.4s)
[4] Create orders ✅ HTTP 200 -- Thank you for your order! Your order
id is 9400 (2.0s)
...output omitted...
```

NOTE

Remember to run the `source ~/.bashrc` command to update your `$PATH` variable and make the `traffic_gen.py` script available.

```
[student@workstation ~]$ source ~/.bashrc
```

5. Inspect the metrics in the OpenShift web console.

5.1. In the workstation machine, open the OpenShift web console in your browser.

Go to `https://console-openshift-console.apps.ocp4.example.com`, click `htpasswd_provider`. Use `developer` as the user and the password.

If you have previously accessed the OpenShift console, then you can be in the Administrator perspective. Change to the Developer perspective. If prompted about the guided tour, then click **Skip tour**.

- 5.2. In the developer perspective, click the **Observe** menu entry, and ensure that the meshobservability-metrics project is selected in the project list dropdown.

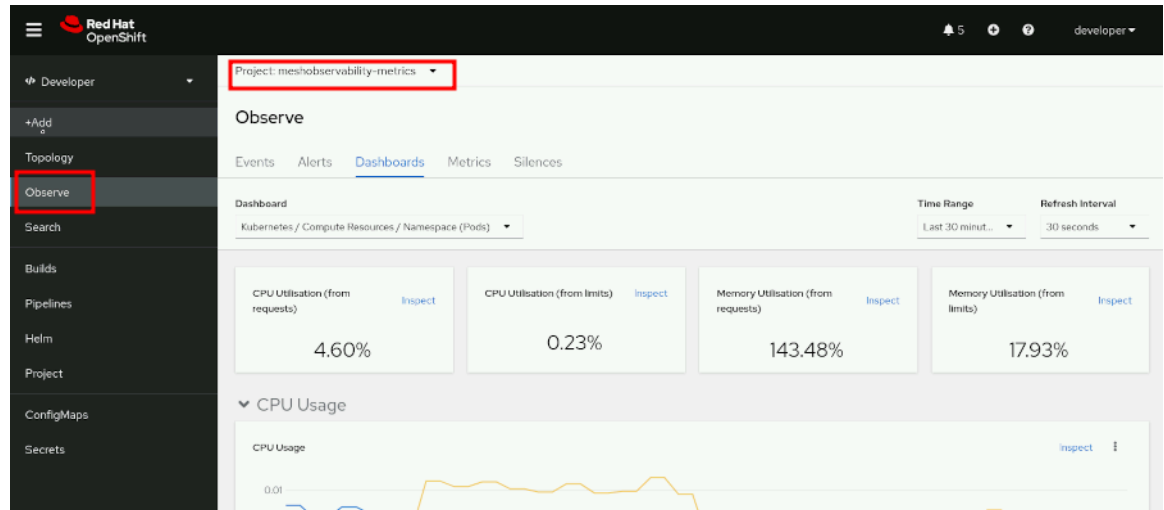


Figure 1.28: The menu Observe in the OpenShift developer console.

- 5.3. Go to the Metrics tab, and, at the bottom of the page, use the query input to search and graph your custom metrics.

In the **Expression** text box, start typing `product_svc:sp150` and select `product_svc:sp150_page_view_count`. Click **Run queries** to see the page view count.

- 5.4. Click **Add query** to add another metric to the same graphic.

In the query text box, start typing `orders_`, and select `orders_placed_total`. Click **Run queries** to view both metrics in the graph.

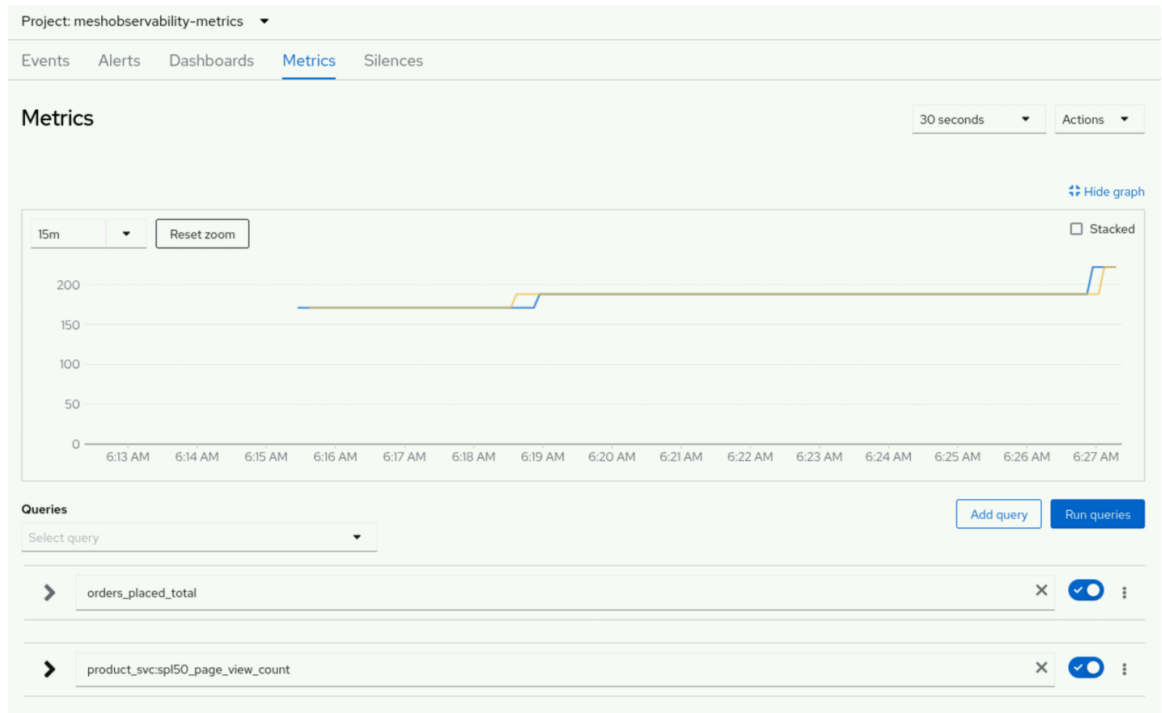


Figure 1.29: Graphic including metrics from the two different microservices.

5.5. Stop the `traffic_gen.py` utility if it is still running.

Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish meshobservability-metrics
```

Revision: ad0007l-3.1-ed3-20260313-9c0e3a4



개인 정보 취급 방침 (https://www.redhat.com/ko/about/privacy-policy?extIdCarryOver=true&sc_cid=701f20000001D8QoAAK)

이용 약관 (<https://www.redhat.com/ko/about/terms-use>)

릴리스 노트
(https://docs.redhat.com/en/documentation/red_hat_learning_subscription/1-latest/html-single/red_hat_learning_subscription_release_notes/index)

Red Hat 교육 정책
(<https://www.redhat.com/ko/about/red-hat-training-policies>)

모든 정책 및 지침
(<https://www.redhat.com/ko/about/all-policies-guidelines>)

쿠키 기본설정